

Konstanta & Struct

```
const NMAX : integer = 100
```

```
ContohStruct =  
type contohdata <  
  nama : string  
  nim : string  
  ipk : integer  
  eprt : integer  
>
```

```
ContohArray =  
type contoharraydata : array[0..NMAX-1] of contohdata
```

Tambah Data

```
procedure tambahData(in/out A : contoharraydata,  
in/out n : integer)  
  algoritma  
    if n < NMAX then  
      input(A[n].field1)  
      input(A[n].field2)  
      input(A[n].field3)  
      input(A[n].field4)  
      n <- n + 1  
    else  
      output("Array Penuh")  
    endif  
  endprocedure
```

Tampil Data (Display)

```
procedure tampilData(A : contoharraydata, n : integer)
kamus
i : integer
algoritma
for i <- 0 to n-1 do
output(
A[i].field1,
A[i].field2,
A[i].field3,
A[i].field4
)
endfor
endprocedure
```

Hapus Data Berdasarkan Pencarian

```
procedure hapusDataCari(in/out A : contoharraydata,  
in/out n : integer,  
x : string)  
kamus  
i, idx : integer  
algoritma  
idx <- -1  
i <- 0  
while i < n and idx = -1 do  
if A[i].field1 = x then  
idx <- i  
endif  
i <- i + 1  
endwhile  
if idx != -1 then  
for i <- idx to n-2 do  
A[i] <- A[i+1]  
endfor  
n <- n - 1  
endif  
endprocedure
```

Ubah Data

```
procedure ubahData(in/out A : contoharraydata,  
idx : integer)  
  algoritma  
    input(A[idx].field1)  
    input(A[idx].field2)  
    input(A[idx].field3)  
    input(A[idx].field4)  
  endprocedure
```

Cari Nilai Ekstrem Maksimum

```
function cariMaks(A : contoharraydata, n : integer) -> integer
{mengembalikan indeks data yang memiliki nilai field terbesar}
kamus
i, idx : integer
algoritma
idx <- 0
for i <- 1 to n-1 do
if A[idx].field3 < A[i].field3 then
idx <- i
endif
endfor
return idx
endfunction
```

Cari Nilai Ekstrem Minimum

```
function cariMin(A : contoharraydata, n : integer) -> integer
{mengembalikan indeks data yang memiliki nilai field terkecil}
kamus
i, idx : integer
algoritma
idx <- 0
for i <- 1 to n-1 do
if A[idx].field3 > A[i].field3 then
idx <- i
endif
endfor
return idx
endfunction
```

Sequential Search (String)

```
function sequentialSearch(A : contoharraydata,  
n : integer, x : string) -> integer  
{mengembalikan indeks data yang dicari, atau -1 jika tidak ditemukan}  
kamus  
i : integer  
algoritma  
i <- 0  
while i < n and A[i].field1 != x do  
  i <- i + 1  
endwhile  
if i < n then  
  return i  
else  
  return -1  
endif  
endfunction
```


Sequential Search (Integer)

```
function sequentialSearch(A : contoharraydata,  
n : integer, x : integer) -> integer  
kamus  
i : integer  
algoritma  
i <- 0  
while i < n and A[i].field3 != x do  
i <- i + 1  
endwhile  
if i < n then  
return i  
else  
return -1  
endif  
endfunction
```

Sequential Search dengan Found

```
function sequentialSearch(A : contoharraydata,  
n : integer, x : string) -> integer  
kamus  
i, found : integer  
algoritma  
found <- -1  
i <- 0  
while i < n and found = -1 do  
  if A[i].field1 = x then  
    found <- i  
  endif  
  i <- i + 1  
endwhile  
return found  
endfunction
```

Binary Search (data harus terurut)

```
function binarySearch(A : contoharraydata,  
n : integer, x : integer) -> integer  
kamus  
left, right, mid, found : integer  
algoritma  
left <- 0  
right <- n - 1  
found <- -1  
while left <= right and found = -1 do  
mid <- (left + right) div 2  
if x < A[mid].field3 then  
right <- mid - 1  
else if x > A[mid].field3 then  
left <- mid + 1  
else  
found <- mid  
endif  
endwhile  
return found  
endfunction
```

Selection Sort Ascending (Kecil -> Besar)

```
procedure selectionSortAsc(in/out A : contoharraydata, n : integer)
kamus
pass, idx, j : integer
temp : contohdata
algoritma
for pass <- 1 to n-1 do
idx <- pass - 1
for j <- pass to n-1 do
if A[idx].field3 > A[j].field3 then
idx <- j
endif
endfor
temp <- A[idx]
A[idx] <- A[pass-1]
A[pass-1] <- temp
endfor
endprocedure
```

Selection Sort Descending (Besar -> Kecil)

```
procedure selectionSortDesc(in/out A : contoharraydata, n : integer)
kamus
pass, idx, j : integer
temp : contohdata
algoritma
for pass <- 1 to n-1 do
idx <- pass - 1
for j <- pass to n-1 do
if A[idx].field3 < A[j].field3 then
idx <- j
endif
endfor
temp <- A[idx]
A[idx] <- A[pass-1]
A[pass-1] <- temp
endfor
endprocedure
```

Insertion Sort Ascending (Kecil -> Besar)

```
procedure insertionSortAsc(in/out A : contoharraydata, n : integer)
kamus
pass, j : integer
temp : contohdata
algoritma
for pass <- 1 to n-1 do
temp <- A[pass]
j <- pass
while j > 0 and temp.field3 < A[j-1].field3 do
A[j] <- A[j-1]
j <- j - 1
endwhile
A[j] <- temp
endfor
endprocedure
```

Insertion Sort Descending (Besar -> Kecil)

```
procedure insertionSortDesc(in/out A : contoharraydata, n : integer)
kamus
pass, j : integer
temp : contohdata
algoritma
for pass <- 1 to n-1 do
temp <- A[pass]
j <- pass
while j > 0 and temp.field3 > A[j-1].field3 do
A[j] <- A[j-1]
j <- j - 1
endwhile
A[j] <- temp
endfor
endprocedure
```

Selection Sort untuk Dua Array Berkorespondensi

```
procedure selectionSortAsc(in/out A, B, n)
kamus
pass, idx, j : integer
tempA, tempB
algoritma
for pass <- 1 to n-1 do
idx <- pass - 1
for j <- pass to n-1 do
if A[idx] > A[j] then
idx <- j
endif
endfor
tempA <- A[idx]
A[idx] <- A[pass-1]
A[pass-1] <- tempA
tempB <- B[idx]
B[idx] <- B[pass-1]
B[pass-1] <- tempB
endfor
endprocedure
```


Insertion Sort untuk Dua Array Berkorespondensi

```
procedure insertionSortAsc(in/out A, B, n)
kamus
pass, j : integer
tempA, tempB
algoritma
for pass <- 1 to n-1 do
tempA <- A[pass]
tempB <- B[pass]
j <- pass
while j > 0 and tempA < A[j-1] do
A[j] <- A[j-1]
B[j] <- B[j-1]
j <- j - 1
endwhile
A[j] <- tempA
B[j] <- tempB
endfor
endprocedure
```

Input Data dengan Sentinel

```
procedure bacaData(in/out A : contoharraydata,  
in/out n : integer)  
kamus  
stop : string  
algoritma  
n <- 0  
input(stop)  
while stop != "STOP" and n < NMAX do  
A[n].field1 <- stop  
input(A[n].field2)  
input(A[n].field3)  
input(A[n].field4)  
n <- n + 1  
input(stop)  
endwhile  
endprocedure
```

Hitung Total (Accumulator)

```
function hitungTotal(A : contoharraydata,  
n : integer) -> integer  
kamus  
i, total : integer  
algoritma  
total <- 0  
for i <- 0 to n-1 do  
total <- total + A[i].field3  
endfor  
return total  
endfunction
```

Rata-Rata

```
function rataRata(A : contoharraydata,  
n : integer) -> real  
kamus  
i : integer  
sum : real  
algoritma  
sum <- 0  
for i <- 0 to n-1 do  
sum <- sum + A[i].field3  
endfor  
return sum / n  
endfunction
```

Menghitung Berdasarkan Kategori

```
for i <- 0 to n-1 do
  if A[i].field3 < batas1 then
    kategori <- "A"
  else if A[i].field3 < batas2 then
    kategori <- "B"
  else
    kategori <- "C"
  endif
endfor
```

Menghitung Dua Kelompok Data

```
total1 <- 0
total2 <- 0
for i <- 0 to n-1 do
  if kondisi then
    total1 <- total1 + A[i].field3
  else
    total2 <- total2 + A[i].field3
  endif
endfor
```

CARA CEPAT: Sort & Search

```
-- Selection Sort Ascending:
if A[idx].field3 > A[j].field3 then idx <- j endif

-- Selection Sort Descending:
if A[idx].field3 < A[j].field3 then idx <- j endif

-- Insertion Sort Ascending:
while j > 0 and temp.field3 < A[j-1].field3 do

-- Insertion Sort Descending:
while j > 0 and temp.field3 > A[j-1].field3 do

-- Sequential Search (cara gampang):
i <- 0
while i < n and A[i].field != x do
i <- i + 1
endwhile
if i < n then return i else return -1 endif
```